



北京航空航天大学
B E I H A N G U N I V E R S I T Y

遥感实验 1-4

姓名：何宗霖

学号：23374275

学院：宇航学院

班级：231516

2025 年 10 月 25 日

一、实验原理

(一) 实验 1——ENVI 软件操作

1、安装 ENVI5.3

(1) 详细操作见 <https://www.cnblogs.com/xingyi-qjl/p/9043636.html>

(2) 需要的 license 在"D:\RS\ENVI\ENVI5.3 软件安装包和安装说明\ENVI5.3 软件安装包和安装说明\ENVI5.3 软件安装包\破解 license\ENVI5.3 破解\Exelis\License\license5.3.lic", 与教程不同

2、破解 ENVI5.3

(1) 需要的 Idl.dll 在"D:\RS\ENVI\文件\经常被隔离的文件\idl.dll", 与教程不同。将此文件替换 Exelis\IDL85\bin\bin.x86_64 路径下的相关文件

(2) 破解完成后将有 ENVI5.3 (64bits) 和 ENVI_Classic5.3 (64bits) 两个程序可以使用, 本人测试后发现, ENVI 5.3 (64-bit)在显示图像上更为方便, ENVI_Classic5.3 (64bits) 在处理图像上 Basic Tool 更加好用, 并且对屏幕占用面积更小, 更加简洁。

3、具体操作 ENVI5.3

(1) 任务 1 图像显示

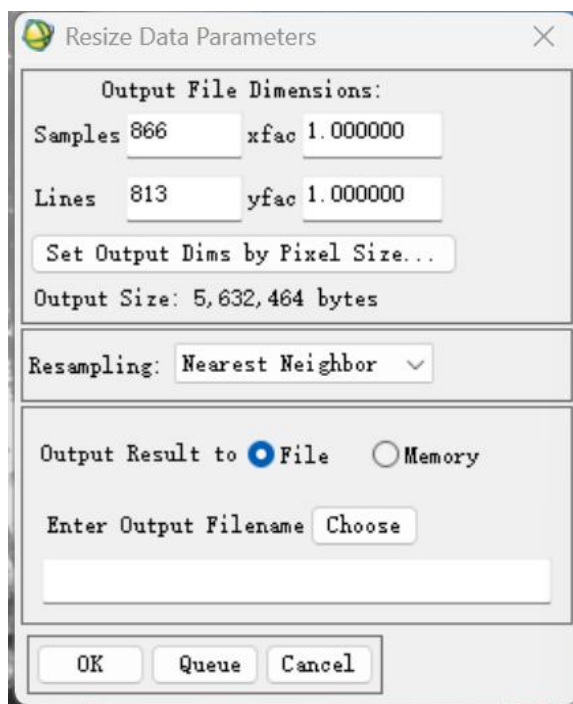
File-Open_Image_File-选择 band-Load_Band

(2) 任务 2 图像大小调整

方法一: 图像显示完成后, 自动跳出三个窗口 (Scoll/Image/Zoom) 其中 Image 是 400*400 大小的窗口; Scoll 是 256*240 大小的窗口; Zoom 是 200*200 大小的窗口



方法二：Basic_Tools-Resize_Data-修改 xfac 和 yfac-重现显示修改并保存后的图像

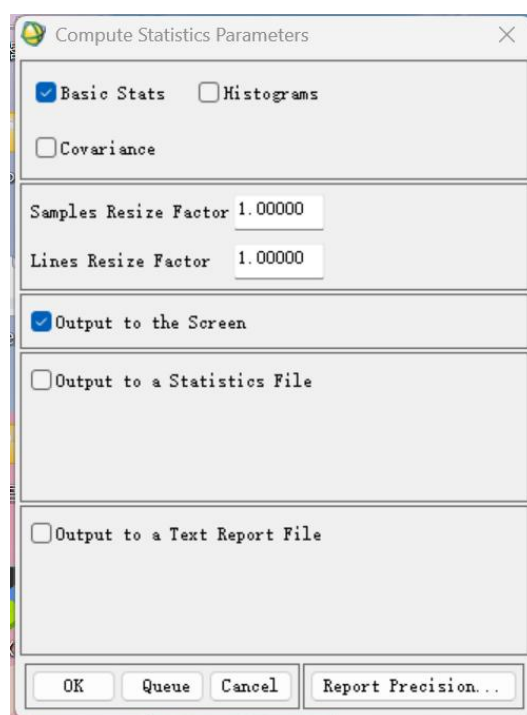


(3) 任务 3 图像信息统计

Basic_Tools-Statistics-Computing_Statistics-

选择 Basic_States 基本统计量+Histograms 直方图+Covariance 协方差-

Select_Plot-Histograms_All-bands

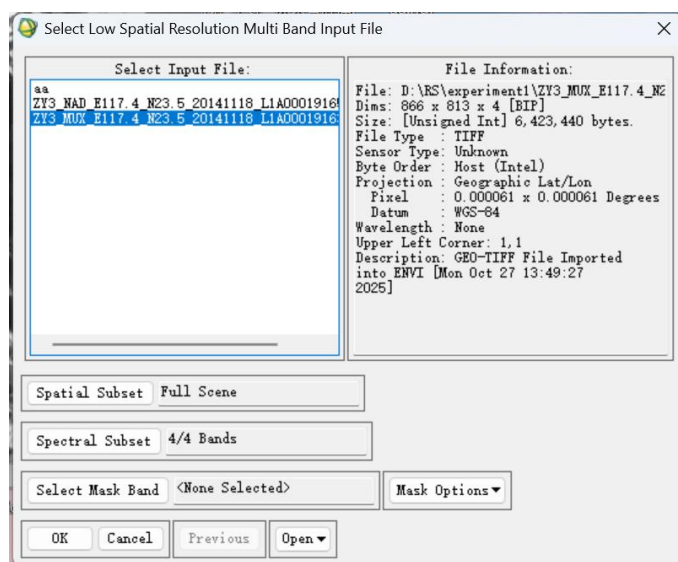


(4) 任务 4 多源图像融合

Spectral-PC_Spectral_Sharpener (Principal Components) /CN_Spectral_Sharpener (Color Normalized)

先选择多光谱图像（至少包含两个光谱）-再选择高分辨力图像-

重现显示修改并保存后的图像



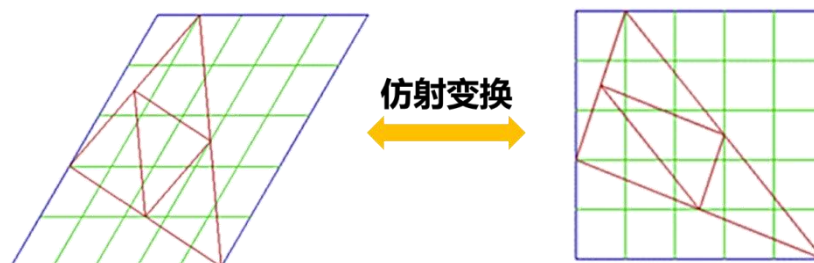
(5) 任务 5 图像异常目标检测

Spectral-RX_Anomaly_Detection-只能选择多光谱的图像

(二) 实验 2——遥感图像几何校正

1、仿射变换概念

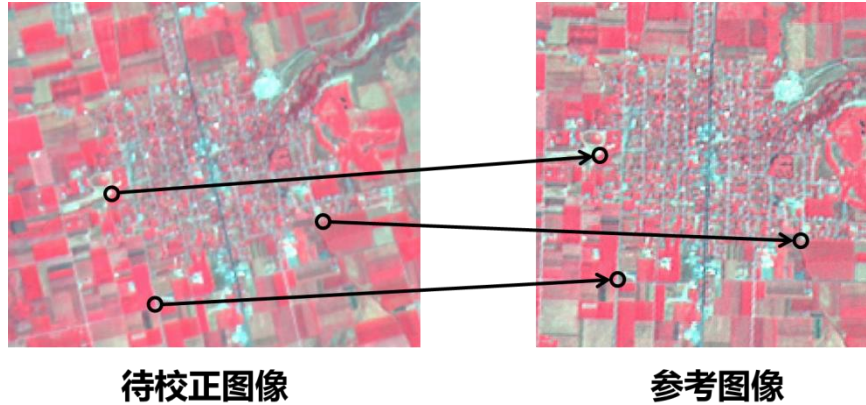
仿射变换指利用待校正图像和参考图像之间的配准关系，通过简单的平移、旋转和剪切等操作，将待校正图像变换到参考图像的坐标系上。



2、仿射变换流程

- (1) 输入待校正图像和参考图像；
- (2) 在待校正图像上选取几个位置点，并在参考图像中找到与之对应的位置点；

- (3) 根据选取点对计算变换矩阵；
- (4) 根据变换矩阵进行仿射变换；
- (5) 输出校正后的结果。



3、实现代码

- (1) 手动选点，按下“enter”截止

```
def select_points_manually(image):
    points = []
    image_display = image.copy()

    def mouse_callback(event, x, y, flags, param):
        nonlocal points
        if event == cv2.EVENT_LBUTTONDOWN:
            points.append([x, y]) # 记录点坐标
            cv2.circle(image_display, (x, y), 5, (0, 255, 0), -1) # 画点(绿色实心圆)
            cv2.putText(image_display, f'{len(points)}', (x+10, y-10),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2) # 标号:1,2,...
            cv2.imshow('Select Points (Click to select, Press Enter to confirm)', image_display) # 显示更新后的图像
            print(f"Selected point {len(points)}: ({x}, {y})") # 打印选中的点坐标

    cv2.namedWindow('Select Points (Click to select, Press Enter to confirm)') # 创建窗口
    cv2.setMouseCallback('Select Points (Click to select, Press Enter to confirm)', mouse_callback) # 设置鼠标回调函数
    cv2.imshow('Select Points (Click to select, Press Enter to confirm)', image_display) # 显示图像

    while True:
        key = cv2.waitKey(1) & 0xFF
        if key == 13: # 检查是否按下了回车键
            break
    cv2.destroyAllWindows()
    return np.array(points, dtype=np.float32) # 返回选中的点坐标，类型为 float32
```

(2) 基于 SIFT 实现特征点匹配【注】使用 KD_tree

```
def find_corresponding_points(image1, image2, src_points):

    # 初始化 SIFT 检测器
    # 是用来检测和描述图像中的局部特征点的算法
    sift = cv2.SIFT_create()

    # 检测关键点和描述子（所有）
    kp1, des1 = sift.detectAndCompute(image1, None)
    kp2, des2 = sift.detectAndCompute(image2, None)

    # 使用 FLANN 匹配器
    # 是用来高效地匹配大量特征点的算法
    FLANN_INDEX_KDTREE = 1
    # 创建索引参数和搜索参数
    index_params = dict(algorithm=FLANN_INDEX_KDTREE, trees=5) # 指定使用 KD 树算法

    # # 更换算法
    # FLANN_INDEX_LSH = 6
    # index_params = dict(algorithm=FLANN_INDEX_LSH) # 使用 LSH 算法, “局部敏感哈希”

    search_params = dict(checks=50) # 指定搜索次数
    flann = cv2.FlannBasedMatcher(index_params, search_params) # 创建 FLANN 匹配器实例
    matches = flann.knnMatch(des1, des2, k=2) # k=2 表示返回每个描述子最好的两个匹配(一个是最近邻, 一个是次近邻)

    # 应用 Lowe's 比率测试筛选良好匹配
    good_matches = []
    for m, n in matches:
        if m.distance < 0.7 * n.distance:
            good_matches.append(m)

    # good_matches = [m for m, n in matches if m.distance < 0.7 * n.distance]

    # 构建匹配点字典: 查询图 (image1) 的关键点索引 -> 训练图 (image2) 的关键点索引
    query_to_train = {m.queryIdx: m.trainIdx for m in good_matches}
```

(3) 主函数

```
def main():
    # 读取图像
    image1 = cv2.imread('D:\\RS\\experiment1\\1.2\\raw.jpg')
    image2 = cv2.imread('D:\\RS\\experiment1\\1.2\\target.jpg')

    # 在第一幅图像中选择点
    print("Select points in the first image.")
    src_points = select_points_manually(image1)

    # 在第二幅图像中寻找对应点
    print("Finding corresponding points in the second image...")
    dst_points = find_corresponding_points(image1, image2, src_points)

    # 计算仿射变换矩阵
    M = cv2.getAffineTransform(src_points[:3], dst_points[:3])

    # 应用仿射变换
    rows, cols, _ = image1.shape
    image3 = cv2.warpAffine(image1, M, (cols, rows))

    # show the result
    cv2.imshow('Original Image1', image1)
    cv2.imshow('Reference Image2', image2)
    cv2.imshow('Transformed Image', image3)
    cv2.waitKey(0)
    cv2.destroyAllWindows()

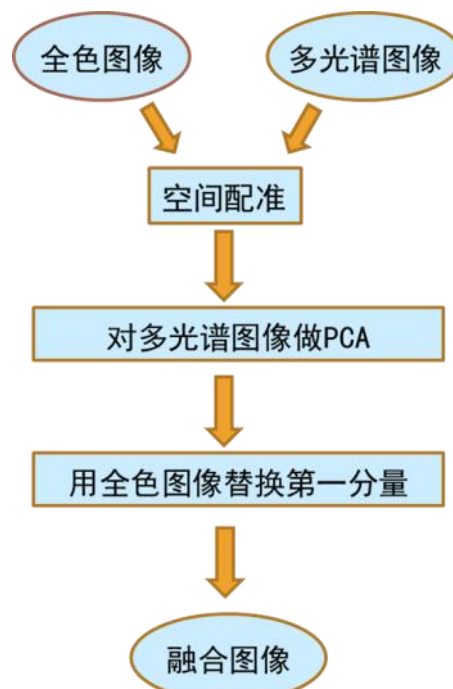
    # Save the result
    cv2.imwrite('D:\\RS\\experiment1\\1.2\\output.jpg', image3)
```


(三) 实验 3——遥感图像融合

1、主成分变换 (PCA)

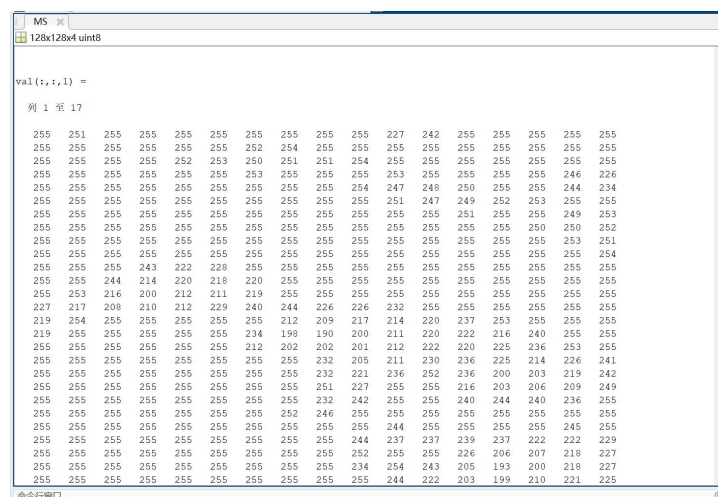
- (1) 输入数据：包括高分辨率的全色图像和低分辨率的多光谱图像。
- (2) 空间配准：将两幅图像对齐，确保它们的空间位置匹配。
- (3) 主成分分析 (PCA)：对多光谱图像进行分析，提取主要成分。
- (4) 替换并融合：用全色图像替换第一主成分，再通过逆变换重构图像

2、图像融合流程



3、原始图像数据

保存在 Matlab 中的 .mat 文件中



	1	2	3	4	5	6	7	8	9	10	11	12	13
1	102	103	104	107	101	100	98	98	108	103	110	155	139
2	113	96	100	104	111	107	103	108	96	108	104	114	136
3	132	124	109	102	106	104	103	112	103	107	101	104	104
4	126	124	124	118	103	99	102	101	103	106	104	103	108
5	127	118	123	120	137	125	106	101	112	103	108	105	102
6	121	123	127	122	113	123	136	107	109	94	103	105	103
7	126	125	122	127	129	119	116	134	128	114	111	108	109
8	170	145	124	119	115	125	125	118	117	132	125	107	104
9	177	182	173	148	130	120	115	135	120	125	124	133	122
10	178	181	188	183	170	145	127	122	126	124	127	123	125
11	168	172	176	185	181	184	170	148	115	121	125	116	121
12	164	176	184	180	196	184	183	196	174	139	116	125	130
13	167	177	182	169	180	178	177	181	178	182	173	130	115
14	182	179	176	182	169	179	168	170	171	180	185	190	166
15	200	188	176	180	178	173	177	180	180	185	182	183	191
16	201	180	171	174	171	170	176	167	161	177	196	191	186
17	189	169	182	185	178	173	176	166	160	182	192	186	192
18	202	184	179	198	182	190	176	166	175	185	195	195	197
19	213	179	178	186	174	185	189	172	181	187	191	193	180
20	212	207	208	202	189	175	185	190	208	211	191	180	184
21	211	197	204	211	209	206	201	205	215	217	219	208	190
22	227	219	199	207	214	223	212	211	224	222	222	218	204
23	180	213	222	218	214	208	231	235	228	227	226	228	201
24	173	165	178	209	220	228	231	240	230	231	234	226	221
25	198	196	173	170	183	211	223	242	234	231	234	233	239
26	161	197	215	206	175	162	173	207	205	226	226	219	217
27	148	159	166	176	208	205	168	161	163	183	194	194	202
28	157	161	163	166	190	196	185	179	159	149	152	157	179
29	162	167	162	161	169	153	167	183	191	164	160	163	158
30	128	147	162	171	174	151	149	182	176	179	192	191	173

原始多光谱图像：



4、代码实现

(1) 从 Matlab 的.mat 文件中读取图像

```
from scipy.io import loadmat
import numpy as np
import cv2
import matplotlib.pyplot as plt

def load_mat_image(filepath):
    data = loadmat(filepath)
    # 找到 .mat 文件中存储图像数据的变量名
    # (过滤掉 '__header__', '__version__', '__globals__')
    key = [k for k in data if not k.startswith('__')][0]
    image = data[key]

    # 确保数据类型为 float32 以便进行数学运算
    return image.astype(np.float32)
```


(2) PCA 前的处理

```
def main():
    #-- 读取图像数据 --
    ms_low_res = load_mat_image("D:\\RS\\experiment1\\1.3\\MS.mat")
    pan_high_res = load_mat_image("D:\\RS\\experiment1\\1.3\\PAN.mat")

    # 记录原始 MS 图像的数据范围, 用于最后的结果裁剪
    ms_min_val = np.min(ms_low_res)
    ms_max_val = np.max(ms_low_res)

    # 定义要选取的波段索引
    desired_bands = [0, 1, 2] # 前三个波段为可见光波段 (R,G,B)
    ms_visible = ms_low_res[:, :, desired_bands]
    D = ms_visible.shape[2] # 波段数 (D=3)

    # 尺寸统一
    H_pan, W_pan = pan_high_res.shape
    ms_upsampled = cv2.resize(ms_visible, (W_pan, H_pan), interpolation=cv2.INTER_CUBIC)

    # 三维→二维
    n = H_pan * W_pan
    X = ms_upsampled.reshape((n, D))

    # 每个波段求全图均值
    mu = np.mean(X, axis=0)
    Z = X - mu # (n, D) - (D,) -> (n, D) (Numpy 广播机制)

    # 求协方差矩阵
    # Sigma = np.cov(Z, rowvar=False) # 这是更高效的Numpy内置方法
    Sigma = (Z.T @ Z) / (n - 1) # @表示矩阵乘法, (D, n) @ (n, D) -> (D, D)

    # 求特征值与特征向量
    eigen_values, E = np.linalg.eig(Sigma) # E 是 (D, D)

    # 将特征向量按特征值从大到小排列, 得到变换矩阵
    sort_indices = np.argsort(eigen_values)[-1:] # 获取降序排列的索引
    eigen_values_sorted = eigen_values[sort_indices]
    V = E[:, sort_indices] # V 是 (D, D)
```

(3) PCA 处理

```
# PCA 正变换
Y = Z @ V # Y 是 (n, D) 矩阵, Y的每一列 $y_i$ 是一个主成分

# 成分替代 (核心步骤)
PC1 = Y[:, 0] # 第一主成分 (n,)
pan_flat = pan_high_res.flatten() # (n,) # 将 PAN 图像展平为一维数组

# 统计匹配
mean_pc1 = np.mean(PC1)
std_pc1 = np.std(PC1)
mean_pan = np.mean(pan_flat)
std_pan = np.std(pan_flat)

# P = (Pan - mean(Pan)) * (std(PC1) / std(Pan)) + mean(PC1)
P_matched = (pan_flat - mean_pan) * (std_pc1 / (std_pan + 1e-8)) + mean_pc1

# 替换
Y_prime = Y.copy()
Y_prime[:, 0] = P_matched

# PCA 反变换
V_transpose = V.T
X_fused = (Y_prime @ V_transpose) + mu # Numpy 广播机制处理加回均值

# 重构
fused_image = X_fused.reshape((H_pan, W_pan, D))
```

(四) 实验 4——遥感图像分割

1、最大类间方差法 (OTSU)

- (1) 读取遥感图像，并将 RGB 图像转为灰度图。
- (2) 显示原图和灰度图。
- (3) 获得灰度图尺寸 $M*N$ ，像素均值 m_g 。
- (4) 遍历包含两个阈值的阈值向量 $T=[T_1, T_2]$ ，遍历范围均为灰度图中最小灰度值到最大灰度值。
- (5) 根据当前遍历的阈值将图像分为三类 C_1, C_2 和 C_3 。
- (6) 计算 C_1 类的像素占全部像素的比例 P_1 ，像素均值 m_1 。
- 计算 C_2 类的像素占全部像素的比例 P_2 ，像素均值 m_2 。
- 计算 C_3 类的像素占全部像素的比例 P_3 ，像素均值 m_3 。
- (7) 根据以下公式计算当前的类间方差。

$$g = \sum_{k=1}^c p_k (m_k - m_G)^2, \quad \text{其中 } c \text{ 是类别数}$$

- (8) 当前类间方差最大时，更新最大方差和选取的阈值，直到遍历结束。
- (9) 根据最终选取的阈值，对图像进行分割，并用不同的灰度值进行表示，显示图像的分割结果。

2、原始图像



3、原始图像（灰度图）



4、代码实现

(1) 迭代查找最佳双阈值

```
# Iterate over possible threshold pairs
for t1 in range(min_val, 127, 5):
    for t2 in range(128, max_val + 1, 5):

        # 向量化分类
        mask1 = gray < t1
        mask2 = (gray >= t1) & (gray <= t2)
        mask3 = gray > t2

        c1, c2, c3 = np.sum(mask1), np.sum(mask2), np.sum(mask3)
        if c1 == 0 or c2 == 0 or c3 == 0:
            continue

        s1, s2, s3 = np.sum(gray[mask1]), np.sum(gray[mask2]), np.sum(gray[mask3])
        m1, m2, m3 = s1/c1, s2/c2, s3/c3
        p1, p2, p3 = c1/total_pixels, c2/total_pixels, c3/total_pixels

        # Calculate between-class variance
        between_variance = p1 * (m1 - global_mean)**2 + p2 * (m2 - global_mean)**2 + p3 * (m3 - global_mean)**2

        # Update best thresholds
        if between_variance > best_variance:
            best_variance = between_variance
            best_thresh_vector = np.array([t1, t2])

t1_start=max(0, best_thresh_vector[0]-10)
t1_end=min(256, best_thresh_vector[0]+10)
t2_start=max(best_thresh_vector[0]+1, best_thresh_vector[1]-10)
t2_end=min(256, best_thresh_vector[1]+10)
```

(2) 最佳阈值附近微调

```
#fine-tune around the best thresholds found
for t1 in range(t1_start, t1_end):
    for t2 in range(t2_start, t2_end):
        if t2 <= t1:
            continue
        count = [0, 0, 0] # number of pixels in each class
        sum_val = np.array([0, 0, 0], dtype=np.int64)
        mask1 = gray < t1
        mask2 = (gray >= t1) & (gray <= t2)
        mask3 = gray > t2
        c1, c2, c3 = np.sum(mask1), np.sum(mask2), np.sum(mask3)
        if c1 == 0 or c2 == 0 or c3 == 0:
            continue
        s1, s2, s3 = np.sum(gray[mask1]), np.sum(gray[mask2]), np.sum(gray[mask3])
        m1, m2, m3 = s1/c1, s2/c2, s3/c3
        p1, p2, p3 = c1/total_pixels, c2/total_pixels, c3/total_pixels
        # Calculate between-class variance
        between_variance = p1 * (m1 - global_mean)**2 + p2 * (m2 - global_mean)**2 + p3 * (m3 - global_mean)**2
        # Update best thresholds
        if between_variance > best_variance:
            best_variance = between_variance
            best_thresh_vector = np.array([t1, t2])

print("Optimal thresholds:", best_thresh_vector)
```

(3) 分割并保存

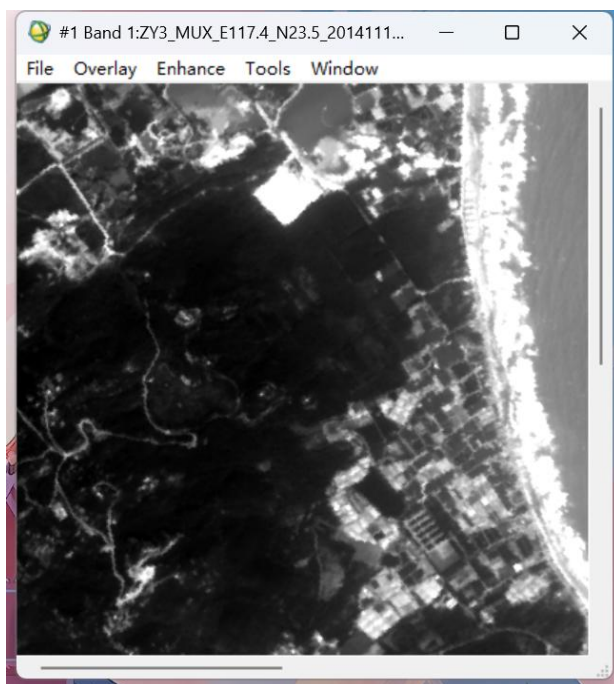
```
# Apply thresholding to create output image
segmented = np.zeros_like(gray)
segmented[(gray >= best_thresh_vector[0]) & (gray <= best_thresh_vector[1])] = 128
segmented[gray > best_thresh_vector[1]] = 255
segmented[gray < best_thresh_vector[0]] = 0

# Save and display result
cv2.imwrite('D:\\RS\\experiment1\\1.4\\output4.jpg', segmented)
cv2.imshow("Original Image", image)
cv2.imshow('Segmented Image', segmented)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

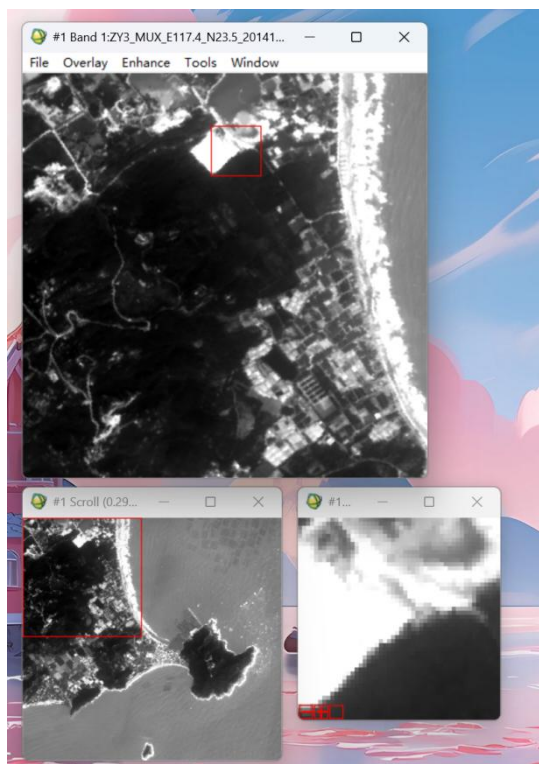
二、实验结果

(一) 实验 1 结果

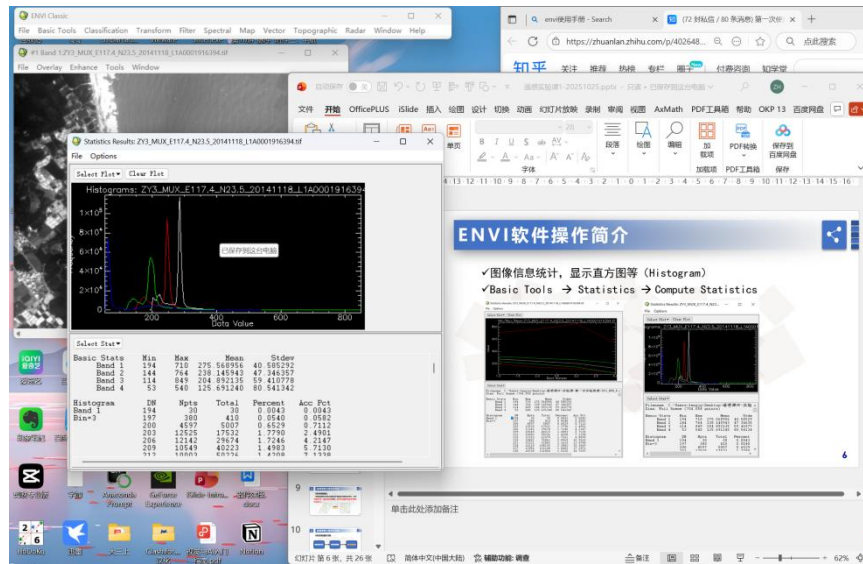
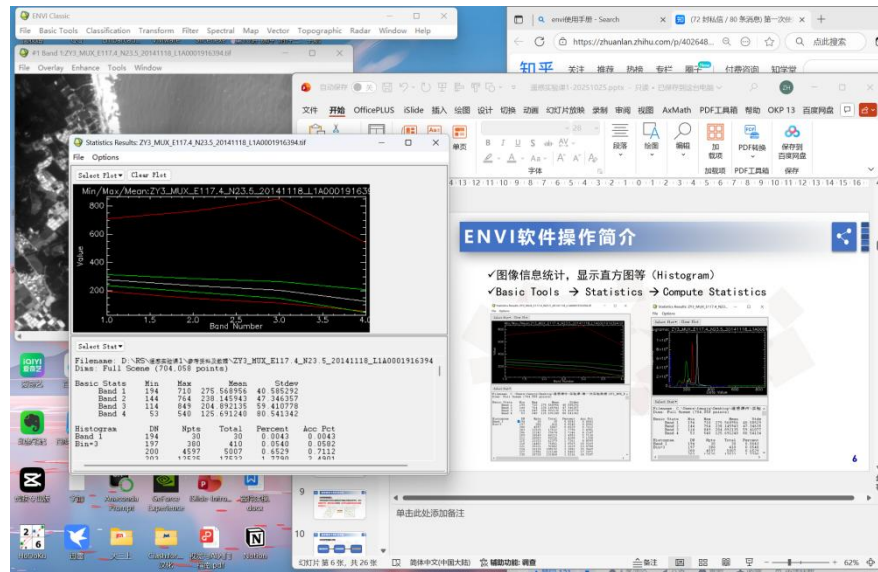
(1) 任务 1 图像显示



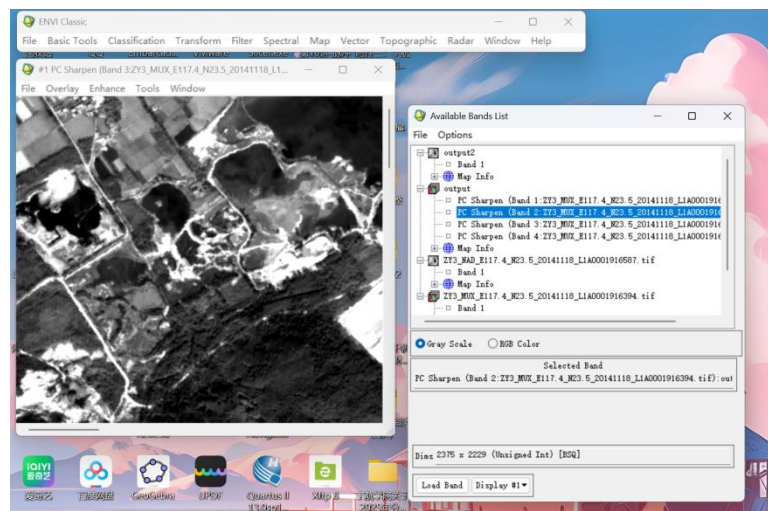
(2) 任务 2 图像大小调整



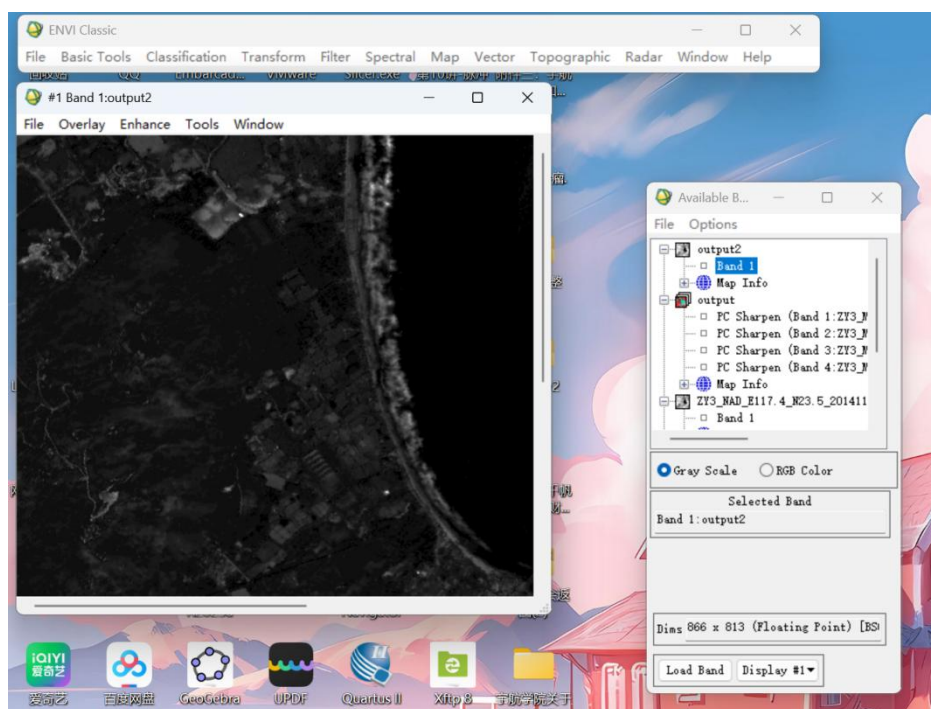
(3) 任务3 图像信息统计



(4) 任务4 多源图像融合

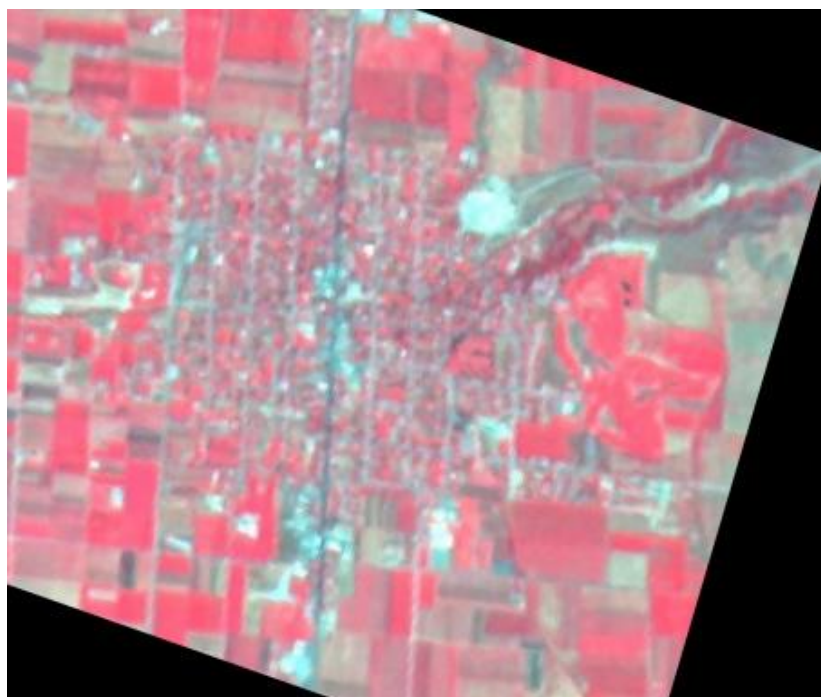


(5) 任务 5 图像异常目标检测

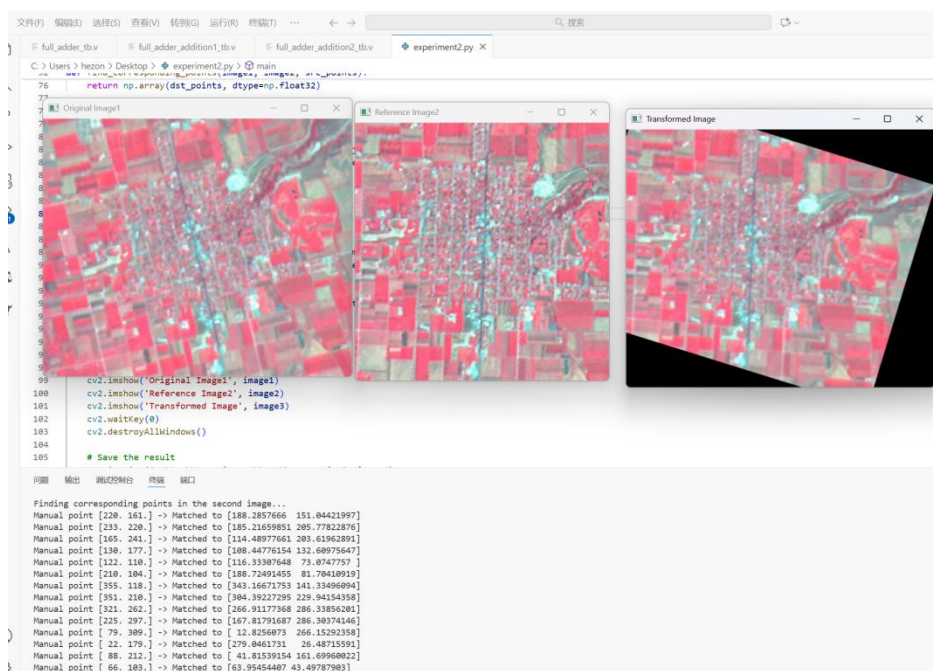


(二) 实验 2 结果

1、校正图像（选择 12 个关键点）



2、VS_code 界面-原始图像+目标图像+校正图像对比



(三) 实验 3 结果

1、使用多光谱图像第 2、3、4 个通道实现融合的图像（可见光波段）



2、使用多光谱图像第 1、2、4 个通道实现融合的图像（更贴近 PPT 里的示例）



3、使用多光谱图像第 1、3、4 个通道实现融合的图像

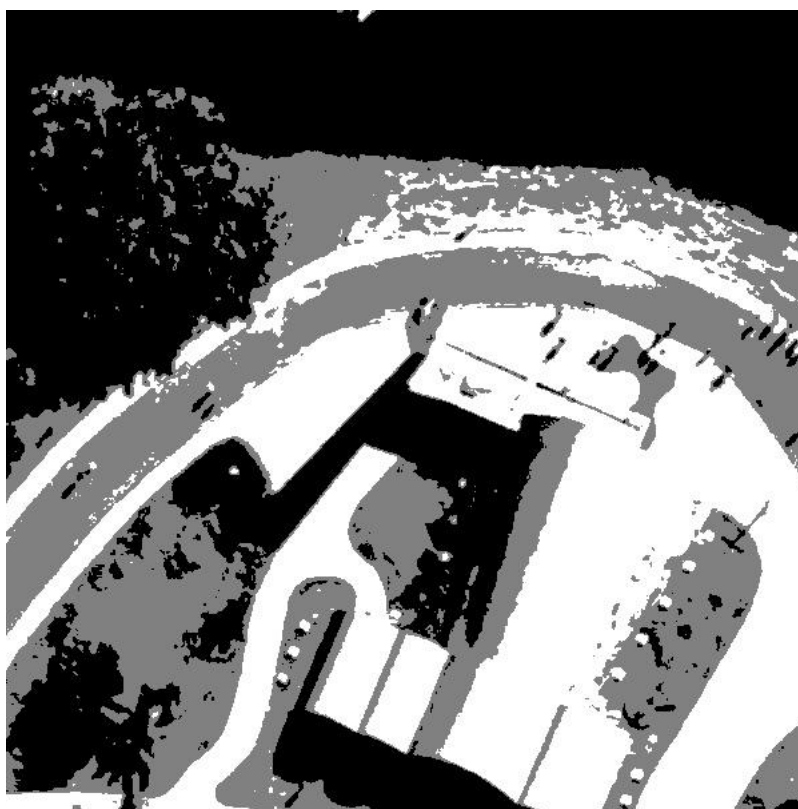


4、使用多光谱图像第 1、2、3 个通道实现融合的图像



(四) 实验 4 结果

1、分割图像



三、实验分析

（一）实验 1

无

（二）实验 2——使用 LSH 算法

(1) 原代码使用 KD 树实现特征点匹配，改用 LSH 算法

```
## 更换算法
# FLANN_INDEX_LSH = 6
# index_params = dict(algorithm=FLANN_INDEX_LSH) # 使用 LSH 算法，“局部敏感哈希”
```

特征	KD 树	LSH
搜索类型	确定性	随机化
匹配精度	较高	较低
查询速度	在高维下性能下降快	极快（时间复杂度与维度无关）
适用数据量	中小型数据集	海量高维数据集
鲁棒性需求	匹配精度高	匹配点噪声大
最终效果	理论上更精确，但耗时较长	速度极快，几何校正结果精度可接受

（三）实验 3——不同波段融合

(1) 组合一：缺少可见光红波段的组合。这种组合的输入是可见光绿波段、可见光蓝波段和红外波段。由于缺失红色信息，融合后的图像在色彩还原上会有所偏差，倾向于青色或蓝绿色调，色彩保真度一般。然而，它保留了人眼敏感的绿色和蓝色信息，同时引入了红外波段。因此，这种组合能有效地突出热目标或伪装目标的显著性，在目标检测能力上表现优秀，但视觉体验略逊。

(2) 组合二：缺少可见光绿波段的组合。这种组合的输入是可见光红波段、可见光蓝波段和红外波段。这种组合会产生最严重的色彩失真。可见光绿波段对人眼的感知最重要，尤其是对自然场景中植被的颜色还原至关重要。

(3) 组合三：缺少可见光蓝波段的组合。这种组合的输入是可见光红波段、可见光绿波段和红外波段。这是综合性能最优的组合。它保留了绿色和红色这两个对人眼最关键的色彩波段，因此融合结果的色彩保真度最高，视觉上最接近自然彩色图像。同时，红外波段的加入使得融合图像在目标显著性和夜视/恶劣环境目标探测能力上得到极大增强。它成功地在保持自然色彩和增加目标信息之间取得了最佳平衡。

(4) 组合四：缺少红外波段的组合。这种组合的输入是可见光红波段、可见光绿波段和可见光蓝波段。融合结果将是原始的彩色可见光图像。它具有无可比拟的最佳色彩保真度和最自然的视觉效果。然而，由于完全排除了红外波段，这种组合无法利用红外信息来增强目标检测能力或提供热力学信息。

（四）实验 4——加速分割

(1) 原始代码，多层循环实现递归——>运行速度慢，运行时间长

```

# # Classify each pixel
# for i in range(height):
#     for j in range(width):
#         v = gray[i, j]
#         if v < t1:
#             count[0] += 1
#             sum_val[0] += v
#         elif v > t2:
#             count[2] += 1
#             sum_val[2] += v
#         else:
#             count[1] += 1
#             sum_val[1] += v

# # Avoid division by zero
# if count[0] == 0 or count[1] == 0 or count[2] == 0:
#     continue

# # Calculate class means
# m1 = sum_val[0] / count[0]
# m2 = sum_val[1] / count[1]
# m3 = sum_val[2] / count[2]

# # Calculate class probabilities
# p1 = count[0] / total_pixels
# p2 = count[1] / total_pixels
# p3 = count[2] / total_pixels

```

(2) 调整后的代码——>使用向量化分类

```

# 向量化分类
mask1 = gray < t1
mask2 = (gray >= t1) & (gray <= t2)
mask3 = gray > t2

c1, c2, c3 = np.sum(mask1), np.sum(mask2), np.sum(mask3)
if c1 == 0 or c2 == 0 or c3 == 0:
    continue

s1, s2, s3 = np.sum(gray[mask1]), np.sum(gray[mask2]), np.sum(gray[mask3])
m1, m2, m3 = s1/c1, s2/c2, s3/c3
p1, p2, p3 = c1/total_pixels, c2/total_pixels, c3/total_pixels

```

(3) 调整前后对比

1. 替换了逐像素的条件判断和分类

循环方式：依赖 CPU 逐个执行 if/elif/else 语句，每次都需要跳转分支。

向量化方式 (mask lines): `gray < t1` 这样的布尔运算是 NumPy 的核心能力。NumPy 在底层将整个数组传递给优化的 C/Fortran/SIMD 函数。CPU 利用 SIMD 指令，在一个时钟周期内，同时对多个像素进行比较操作。直接生成一个与 `gray` 同等大小的布尔掩码。

2. 替换了逐像素的累加计数

循环方式：依赖 CPU 逐个像素执行 `c1 += 1` 这样的标量累加操作。

向量化方式 (`np.sum(mask)`): `np.sum(mask1)` 实际上是计算布尔数组中 True 的个数。这个操作在底层也被高度优化，速度远快于 Python 解释器中的循环累加。

四、实验心得

（一）实验一：ENVI 软件基础操作

初次接触 ENVI 5.3，最直观的感受是其作为专业遥感软件的强大和复杂。从软件的安装配置，到熟悉其“工具箱”的各项功能，这是一个从陌生到基本掌握的过程。这次实验让我认识到，在实际工作中，一款高效的专业工具能够极大地提升效率。同时，通过比较 ENVI 5.3 和 ENVI Classic 的不同操作界面，也体会到不同版本在图像显示和处理功能上的侧重和便利性差异，让我对遥感数据的可视化和基本管理有了初步概念。

（二）实验二：基于 LSH 算法的几何校正

通过这次实践，我对仿射变换的几何意义有了更深的理解。它能够涵盖平移、旋转、缩放和剪切等多种线性几何失真。我体会到，仿射变换作为一阶多项式模型，其优点是计算简单、运算速度快，适用于处理变化相对均匀的几何畸变。但其精度高度依赖于所选取匹配点的数量和分布，点选得越多、分布越均匀，校正结果就越可靠。

（三）实验三：多波段融合与分析

多波段融合实验让我对多光谱数据的物理意义有了更直观的理解。通过尝试不同的波段组合，我亲眼看到了图像色彩是如何随波段分配而改变的。

（四）实验四：图像分割

图像分割实验是将遥感数据从“像素级”提升到“目标级”认知的关键一步。无论是基于阈值还是更复杂的区域生长算法，其核心都是将具有相似特征的相邻像素聚合起来，形成具有实际意义的区域。在实际操作中，调节分割参数的细微变化，都会对最终的分割结果产生巨大影响。